

MITI

La rifa del grupo, sin planillas ni discusiones

Entrega final de proyecto · Inteligencia Artificial aplicada a organizaciones

Curso de Inteligencia Artificial para Programadores

Universidad Tecnológica Nacional · Facultad Regional Buenos Aires

Integrante: Ezequiel Bari — análisis, arquitectura, orquestación de agentes, infraestructura y pruebas.

Links de acceso directo

El docente evalúa en estos links. Todos funcionan al momento de la entrega.

Recurso	URL
Repositorio (público)	https://github.com/lechu-sg/miti
Sitio en producción	https://miti.sole.ar
APK para instalar	https://miti.sole.ar/descargas/miti.apk
Video de demostración (2:40)	https://miti.sole.ar/descargas/entrega/miti-demo.mp4
Capturas de pantalla	https://miti.sole.ar/descargas/entrega/
Panel de administración	https://miti.sole.ar/admin
Definición del sistema	github.com/lechu-sg/miti/blob/main/docs/DEFINICION.md
Traspaso entre agentes	github.com/lechu-sg/miti/blob/main/docs/TRASPASO.md
Anexo de IA local	github.com/lechu-sg/miti/blob/main/docs/ANEXO_IA_LOCAL.md

Acceso de evaluación al panel: Para evaluar el panel de administración (<https://miti.sole.ar/admin>), el docente dispone de la cuenta **cursos.agt@gmail.com**. Al ingresar el correo se envía un código de 6 dígitos por email y en el primer acceso se solicita vincular el segundo factor TOTP escaneando el código QR con Google Authenticator u otra app autenticadora.

Parte 1 · El proyecto como aplicación real

1. Presentación del proyecto

Qué es. Miti es una aplicación Android para administrar una campaña de recaudación en grupo: una rifa con talonario de números o una venta de productos. Cada integrante registra lo que vende y, sobre todo, **dónde quedó la plata**.

El problema. Cuando un grupo junta plata —el viaje de egresados, el club, la cooperadora— el dinero termina repartido: una parte en la cuenta de quien recibe las transferencias, otra en la billetera virtual de cada uno y otra en efectivo. Alguien adelantó las fotocopias y hay que devolvérselo. La cuenta la lleva una sola persona en una planilla que nadie más toca, y al repartir empieza la discusión: cuánto entró, cuánto se gastó y cuánto le toca a cada uno.

La solución implementada. La app sabe en todo momento cuánto hay en cada caja, qué falta cobrar y quién debe. Los gastos requieren la aprobación de otro integrante; nadie se aprueba a sí mismo. Al cerrar, calcula el neto, lo divide en partes iguales y resuelve **las transferencias mínimas** para saldar al grupo.

Público objetivo. Grupos de 3 a 50 personas sin conocimiento contable ni técnico: familias, cursos, clubes de barrio, equipos de baile. Gente que hoy usa WhatsApp y papel. La app se instala por link, sin pasar por Play Store.

2. Arquitectura

2.1 · Cómo fluyen los datos

El celular es autónomo: guarda en SQLite y encola lo que se registra sin señal. Cuando hay conexión, sincroniza contra la API, que valida las reglas de negocio con transacciones y bloqueos en PostgreSQL. La memoria persistente del sistema vive en PostgreSQL (servidor) y en SQLite (celular); los comprobantes, como archivos en disco.

Qué es IA y qué es lógica tradicional. Todo lo que toca dinero es lógica determinista: son cuentas entre personas y una alucinación no es aceptable. El único componente de IA en el producto es un modelo chico que corre en el mismo servidor y **redacta** el recordatorio de deuda; la persona lo revisa y decide si lo manda.

App Android (Flutter, SQLite + outbox) --HTTPS--> Caddy (TLS) --> API FastAPI
API --> PostgreSQL 18 (memoria persistente) --> pgBackRest --> Oracle + Backblaze
API --> archivos de comprobantes · SMTP (códigos de acceso) · MercadoPago (planes)
API --> Ollama + Llama 3.2 en el mismo VPS (redacción de recordatorios)

El diagrama completo (Mermaid), el flujo de agentes y el diagrama de secuencia de una venta sin señal están en **docs/DEFINICION.md** y en el anexo del repositorio, donde GitHub los dibuja.

2.2 · El orquestador y los agentes

Esta es la parte que da cuenta de lo aprendido en el curso: **el software no se escribió a mano, se orquestó**. Un humano define las reglas y decide; un orquestador (Antigravity y Claude Code) reparte cada etapa —análisis, arquitectura, backend, diseño, infraestructura, pruebas— entre agentes de **dos modelos de dos empresas distintas**, que se pasan el trabajo mediante documentos versionados en el repositorio.

El ciclo: el humano decide → el orquestador lee la memoria (DEFINICION.md, TRASPASO.md, la skill de diseño) → asigna la etapa al agente que corresponde → el agente de pruebas corre las suites contra el servidor real → si falla vuelve al orquestador; si pasa, se despliega y el humano prueba en el celular. Cada vuelta actualiza la memoria.

La memoria persistente del sistema de agentes no son mensajes de chat: son archivos en git. Por eso, cuando se agotó la cuota de un modelo, se escribió un documento de traspaso y el otro continuó desde la fase 2 hasta la 7 sin intervención manual sobre el código.

Evidencia verificable en el historial: 90 commits en 11 días, de los cuales **47 están co-firmados por Claude Opus 5 y 43 por Gemini 3.8 Flash.**

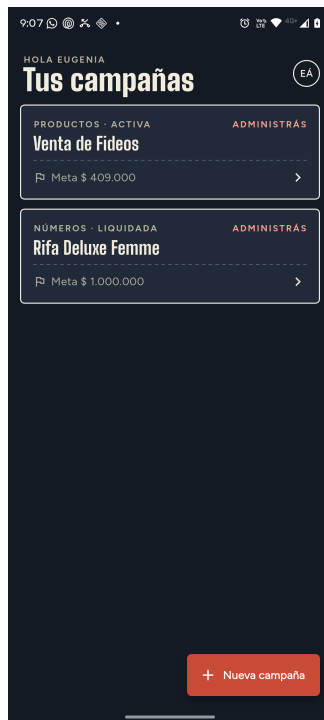
3. Stack tecnológico

Criterio de elección del stack: La selección del ecosistema tecnológico respondió fundamentalmente a dos factores: por un lado, las **recomendaciones de los modelos de IA** sobre mejores prácticas para cada componente (transacciones ACID estrictas, despliegue reproducible en contenedores, persistencia offline-first y backend asíncrono tipado); por el otro, la decisión de **aprovechar el proyecto para explorar terreno desconocido y aprender** interactuando con tecnologías que el autor jamás había utilizado. El aprovisionamiento y administración de un VPS Linux ARM, el motor PostgreSQL con migraciones y copias continuas cifradas, el lenguaje Dart con Flutter para desarrollo móvil, y la API asíncrona con FastAPI en Python representaban territorio nuevo. La asistencia continua de la IA permitió avanzar sobre terreno desconocido con gran facilidad y velocidad de implementación, alcanzando una arquitectura sólida y profesional.

Componente	Tecnología	Por qué esta y no otra
Frontend	Flutter 3.47 (APK Android)	Un solo código, funciona sin señal y se distribuye por link sin pasar por Play Store, que era requisito del proyecto.
Backend	Python + FastAPI (async)	Tipado con Pydantic, documentación automática y el mismo lenguaje para la API, el panel y las tareas de mantenimiento.
Base de datos	PostgreSQL 18	Transacciones y bloqueos reales: sin eso, dos personas venden el mismo número. SQLite quedó del lado del celular.
Base local	SQLite + outbox propio	Permite vender sin señal, que es el caso más común en una rifa de barrio.
Modelo de IA (producto)	Ollama + Llama 3.2 3B, local	Los datos del comprador no pueden viajar a un tercero (Ley 25.326). Corre en el mismo VPS, sin costo por token y sin internet.
Modelos de IA (desarrollo)	Claude Opus 5 y Gemini 3.8 Flash	Dos proveedores para comparar criterios y para no depender de una sola cuota.
Orquestación	Antigravity y Claude Code, memoria en archivos del repo	No hacía falta LangChain: el orquestador ya ejecuta agentes. Lo que faltaba era memoria, y se resolvió con documentos versionados.
Despliegue	Docker Compose + Caddy · VPS Oracle ARM	HTTPS automático, costo cero en el nivel gratuito y control total del dato.
Pagos	MercadoPago Checkout Pro	Es lo que usa el público argentino; el servidor nunca ve datos de tarjeta.
Copias de seguridad	pgBackRest + gpg a Oracle y Backblaze	Dos proveedores distintos, cifradas y con restauración probada automáticamente.

4. Evidencia de funcionamiento

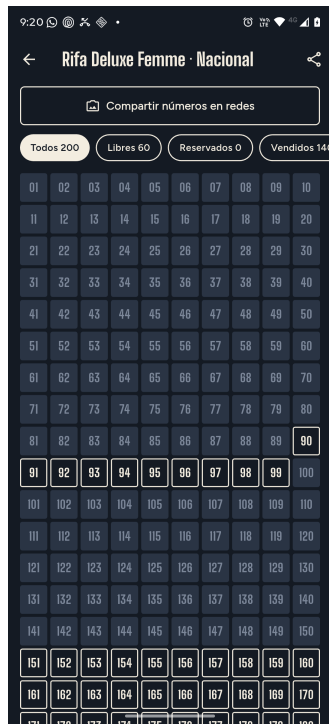
Los datos que se ven son los de una rifa real de 200 números a \$5.000 del grupo *Femme Deluxe Bachata*, cargada íntegra en el servidor de producción: 10 integrantes, 141 ventas, gastos aprobados y la campaña gemela ya liquidada.



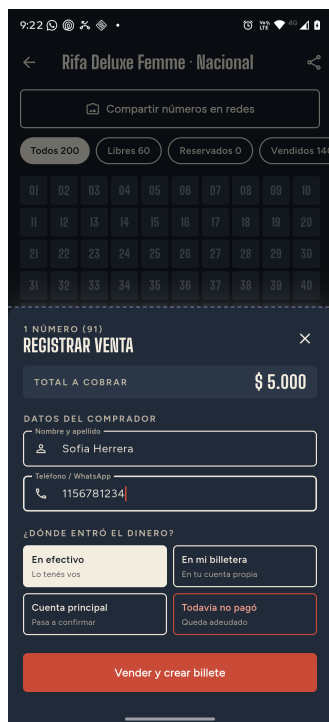
Inicio: las campañas de la usuaria.



La campaña en curso: cobrado, vendido, cuánto falta y los accesos.



El talonario completo: 200 números, con los libres resaltados.



Registrar una venta: quién compró y, sobre todo, dónde entró el dinero.



El billete que se le manda al comprador por WhatsApp.



Dónde está la plata: la cuenta principal y la billetera de cada integrante.



La liquidación: \$937.000 netos, \$93.700 por integrante y las transferencias mínimas para saldar al grupo.



La imagen para redes: la app pega los números libres sobre el flyer del grupo.

Video de demostración

Una sola toma de 2 minutos y 40 segundos con el ciclo completo:
<https://miti.sole.ar/descargas/entrega/miti-demo.mp4>

Registro de una ejecución real

Suites de prueba contra <https://miti.sole.ar> (datos reales, servidor de producción)
fase1 39/39 · fase2 55/55 · productos 43/43 · sync 26/26 · liquidación 34/34
gastos 33/33 · fase5 47/47 · fase6 26/26 · perfil 16/16 · planes 32/32
admin 41/41 · IA local 15/15 => **407 pruebas, 0 fallas**

2026-09-28 00:25:57 INFO miti.ia Recordatorio redactado por llama3.2:3b en 17885 ms
POST /campanas/2e55.../ventas/7a78.../recordatorio HTTP/1.1 200 OK

5. Evaluación UX/UI

Heurística	¿Cumple?	Evidencia
Visibilidad del estado	Sí	Cada venta muestra si está pagada, adeudada o «a confirmar» (registrada sin señal); el ícono de sincronización indica lo pendiente.
Coincidencia con el mundo real	Sí	El lenguaje es el del grupo: talonario, billete, caja, «dónde entró el dinero». Nada de «transacción» ni «entidad».
Control y libertad	Parcial	Se puede anular una venta con aprobación de otro integrante; no hay deshacer inmediato.
Consistencia y estándares	Sí	Sistema de diseño propio («Talonario») escrito como skill obligatoria para los agentes: mismos componentes en todas las pantallas.
Prevención de errores	Sí	Nadie aprueba su propio gasto; la liquidación lista los impedimentos antes de cerrar; un número vendido no se puede vender dos veces, ni siquiera sin señal.
Reconocimiento sobre recuerdo	Sí	El destino del dinero se elige de una lista con los saldos a la vista.
Flexibilidad y eficiencia	Parcial	Se venden varios números en una operación; falta buscador de compradores.
Estético y minimalista	Sí	Una acción principal por pantalla; la publicidad del plan gratis nunca aparece en las pantallas de dinero.
Ayuda a reconocer errores	Sí	Mensajes en criollo («no pudimos conectarnos, ¿tenés señal?») y el error se muestra donde el teclado no lo tapa.
Ayuda y documentación	Parcial	Textos de ayuda en contexto; no hay manual.

5.2 · Prueba con usuario real

Se probó con una usuaria real en un Android 16 y el feedback cambió el producto. En una sola sesión reportó cuatro cosas: el campo de mail no ofrecía autocompletado, el mensaje de error quedaba tapado por el teclado, el aviso era rojo con letra negra (ilegible) y el modo oscuro tenía poco contraste —medido: las líneas estaban en 1,6:1 contra un mínimo razonable de 3:1—.

Las cuatro se corrigieron y, además, se incorporaron como **reglas permanentes en la skill de diseño**, para que ningún agente vuelva a cometerlas. Ese es el punto: el feedback de una usuaria real se convirtió en memoria del sistema de agentes.

6. Ciberseguridad

Riesgo	Tipo	Medida tomada
Robo de la base con datos personales	Privacidad (Ley 25.326)	El email se guarda cifrado con AES-256-GCM; se busca por huella HMAC, nunca en claro.
Robo de credenciales	Autenticación	Sin contraseñas: código de 6 dígitos por mail, token de 15 minutos y refresco rotativo con detección de reuso (si se reusa, cae toda la cadena).
Secretos en el código	Secretos	Ningún secreto versionado: archivos en /srv/miti/secrets inyectados como Docker secrets. Verificado sobre todo el historial antes de abrir el repositorio.
Pago falsificado	Integridad / fraude	El webhook de MercadoPago sólo avisa; el estado se consulta a su API con nuestro token y se validan importe y moneda antes de habilitar el plan.
Toma del panel de administración	Control de acceso	Código por mail más TOTP, cookie firmada de 2 h, token CSRF por formulario, CSP sin scripts y auditoría de cada ingreso, fallo y acción.
Inyección de instrucciones en el modelo	Prompt injection	Los datos del comprador entran como datos, separados de la consigna y recortados; la respuesta se valida (nombre, importe exacto, sin otras cifras) y, si no pasa, se usa la plantilla.
Espiar la pantalla con saldos	Privacidad	FLAG_SECURE opcional: bloquea capturas y oculta la app en la lista de recientes.
Uso de la app para molestar a terceros	Abuso	Tope de 5 códigos por hora por dirección y de envíos por hora; sólo se invita a quien ya tiene cuenta.
Pérdida de datos	Disponibilidad	Copias cifradas diarias en dos nubes distintas, con restauración probada automáticamente cada mes (base y archivos).
Retención indefinida	Privacidad	Borrado automático 6 meses después de liquidada la campaña; baja de cuenta que anonimiza de forma irreversible.

7. IAs usadas en el co-work de desarrollo

Herramienta	Para qué se usó	Aportó bien / mal / sorprendió
Claude Opus 5 (Claude Code)	Orquestación, infraestructura, seguridad, sistema de diseño, pruebas de punta a punta, IA local.	Sorprendió en infraestructura: dejó copias cifradas en dos nubes con restauración probada, algo que a mano se posterga siempre.
Gemini 3.8 Flash (Antigravity)	Fases 2 a 7: rifa, modo sin señal, liquidación, productos, sorteo, avisos, y la pantalla del recordatorio con IA.	Bien en volumen: 43 commits con la app y la API completas siguiendo el documento de traspaso, sin intervención sobre el código.
Llama 3.2 3B (Ollama, local)	Redacta el recordatorio de deuda dentro del producto.	Bien con consignas cerradas; mal en preguntas abiertas: en la prueba inventó un «algoritmo de gestión de loterías» que no existe.



El modelo del servidor redacta el recordatorio; la app avisa que lo escribió la IA y que hay que revisarlo antes de mandarlo.

Reflexión: ¿Eso es programar?

Cruce entre modelos y traspaso: No se utilizó una única IA, sino un cruce complementario entre modelos de distintos proveedores (Claude Opus 5 de Anthropic y Gemini 3.8 Flash de Google), sumando además a Llama 3.2 en local. Claude estructuró la arquitectura inicial, las suites de prueba y la infraestructura de despliegue. Al agotarse la cuota, Gemini tomó el relevo leyendo la memoria persistente en git (*docs/DEFINICION.md* y *docs/TRASPASO.md*), relejendo las decisiones previas y completando 6 fases funcionales sin pérdida de contexto ni retrabajo.

El rol humano y el paradigma de AI coding: Frente a la pregunta formulada en el curso sobre si *pasar de una IA a otra para su relectura y mejora es programar*, la experiencia de este proyecto demuestra que **es la esencia del nuevo paradigma de AI coding y loop coding**. Programar ya no consiste en tipear sintaxis ni pelear con la gramática de un lenguaje; el programador asume el rol de **arquitecto y director de orquesta**: define las reglas de negocio, gobierna la memoria y el contexto, audita rigurosamente los desvíos (como los errores de estilo o de ofuscación que los modelos pasaron por alto) y valida sobre el entorno real. Sin este co-work asistido, la profundidad del sistema —con 407 pruebas automáticas, copias WAL cifradas en dos nubes y doble factor en el panel— hubiera sido inalcanzable en el tiempo disponible para una sola persona.

Parte 2 · IA local en el proyecto

Qué se instaló. Ollama con Llama 3.2 3B (2 GB) en el mismo VPS de la aplicación: Oracle A1, ARM, 4 núcleos, 23 GB de RAM y **sin GPU**. Escucha sólo en la puerta de enlace de la red interna de Docker: no está publicado en internet y el firewall acepta ese puerto únicamente desde esa subred.

1. Qué papel juega el modelo local

Es un **subagente de soporte**, nunca el que decide sobre la plata. Hoy hace una sola cosa: redactar el mensaje con el que un vendedor le recuerda a un comprador que debe su parte. Antes ese texto salía de una plantilla fija, siempre igual. No reemplaza ninguna API externa, porque el producto no usaba IA: **habilita algo que antes no se podía hacer**, porque mandar el nombre y la deuda de un tercero a una nube ajena no es aceptable.

2. Qué le aporta al usuario

Que la app escriba por él. Redactar un mensaje para cobrarle a un vecino es incómodo y por eso la deuda se posterga; que el sistema proponga el texto acelera la cobranza, que es el problema real del grupo. Y sobre todo: **los datos no salen del servidor**. Nombres y teléfonos de compradores son datos de terceros que nunca dieron consentimiento; con el modelo local, esa conversación legal no existe. El costo por token es cero, así que la función puede estar en el plan gratuito.

3. Qué aporta como profesional

Poder mirar lo que hoy no se puede mirar. Están los registros de operación, los movimientos y los avisos, pero analizarlos con una nube implicaría exportar datos personales. Con un modelo local se le puede preguntar al registro en qué pantalla abandona la gente, qué campañas se traban antes de liquidar o qué errores se repiten, sin sacar un byte del servidor. El análisis deja de ser un proyecto y pasa a ser una pregunta.

4. Limitaciones frente a una API en la nube

Hardware: 4 núcleos ARM sin GPU dan ≈ 14 tokens por segundo; un mensaje tarda entre 7 y 18 segundos. Sirve para una acción que la persona pide a propósito, no para algo que se escriba mientras mira la pantalla.

Calidad: un modelo de 3B razona mal en preguntas abiertas. En la prueba de la terminal se inventó un «algoritmo de gestión de loterías» y ni mencionó la privacidad, que era la respuesta correcta. Por eso en el producto sólo recibe consignas cerradas, con ejemplo y datos concretos, y aun así se valida lo que devuelve: si no nombra a la persona, si no dice el importe exacto, si aparece cualquier otra cifra en pesos o si hace preguntas, se descarta y se usa la plantilla.

Mantenimiento: actualizar el modelo, medir que no empeore y sostener las pruebas pasa a ser responsabilidad propia. Por eso el diseño lo deja como accesorio: si el modelo no está, la app funciona igual.

Medición registrada: en la primera prueba, con el modelo todavía inalcanzable, el sistema devolvió la plantilla en 58 ms sin un solo error. Con el modelo disponible, redactó en 17,9 segundos. El detalle completo está en docs/ANEXO_IA_LOCAL.md.

Estado y próximos pasos

Criterio de evaluación	Estado
App funcionando y demostrable	Publicada y en uso: APK descargable, servidor en línea, video y capturas.
Arquitectura documentada	Diagramas en DEFINICION.md (arquitectura, agentes, secuencia) y tabla de stack completa.
Evaluación UX/UI	10 heurísticas evaluadas y prueba con usuaria real, con correcciones aplicadas.
Ciberseguridad	10 riesgos con la medida tomada en cada uno.
Parte 2 · IA local	Modelo corriendo en el VPS, integrado al producto y medido.
Publicación en Google Play Store	No se realizó porque se requiere tener la app dada de alta y aprobada formalmente en Google Play; el proyecto se definió para distribuirse directamente por enlace (APK sin intermediarios ni demoras de revisión). Por esta misma razón, AdMob y notificaciones push operan con identificadores oficiales de prueba de Google.

Lo que sigue. Tramitar la cuenta de desarrollador en Google Play para habilitar AdMob con anuncios de producción y Firebase Cloud Messaging con credenciales reales (no se implementó en esta entrega dado que Google exige tener la aplicación publicada y verificada en su tienda); cargar las credenciales de MercadoPago para habilitar el cobro de planes; y coordinar una prueba cerrada con uno o dos grupos reales antes de mover Miti a su dominio propio definitivo.